1 Lesson 15-10-26

1.0.1 ROM based PLD performance

In the last lesson we began covering PLDs, starting with ROM based PLDs. These are easy to implement, but have memory size requirements growing exponentially with the number of inputs. This makes them slow, with address decoder which become large and very deep in levels of logic.

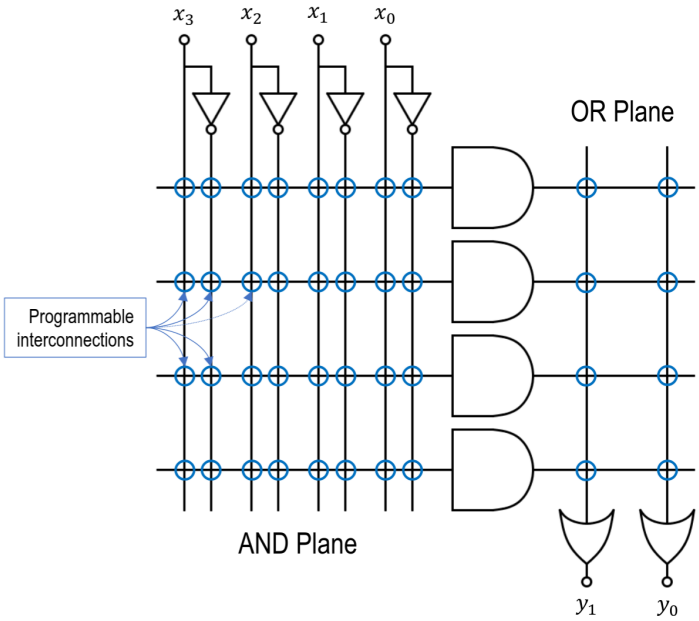
1.1 Programmable Logic Arrays

The alternative approach is given by *Programmable Logic Arrays* (**PLA**s). The basic structure of a ROM based PLD is:

1. The NOT plane, where we get signals and their inverted values;
2. The AND plane, which ands the signals from the NOT plane;
3. The final OR sum on every output.

This is the basic **SoP** (*Sum of Products*) architecture.

A programmable Logic Array is just a circuit made up of programmable AND and OR planes, where the interconnections are designed by the user.



An m-input n-output PLA with k AND gates (each with k inputs), and $k < 2^m$, can implement any set of n combinational logic functions of m inputs, provided that the total number of implicants (cubes) is not larger than k, the number of available AND gates. Note that a ROM can implement any function as it has $k = 2^m$ AND gates.

This is usually not too much of an issue, plus it's really efficient:

- PLA size linearly depends on the number of inputs: the number of AND gates k does not depend on the number of inputs m (it is only restricted by it, should it become a PLD). This means the PLA size linearly depends on the number of AND gates;
- ROM size is exponential on the number of inputs.

The number of AND gates (k) is decided by the manufacturer based on constraints, like performance, costs, etc...

It is interesting to see how the **programmable interconnections** are designed. All intersections are programmable interconnections, and the programming techniques are basically the same as memories:

- For the OR Plane, we usually use wired OR logic similar to memories, programmed via fuses or floating gate transistors;
- For the AND Plane, we similarly use fuses or floating gate transistors.

1.1.1 PLA performance

Let's evaluate the performance of PLAs. Speed is low, as in ROMs (if not slower), due to the several programmable interconnections: we have high parasitic resistances and capacitances.

The problem worsens when implementing sequential functions, where the output is in feedback to the input, *outside* the integrated circuit.

Lastly, by increasing the number of input and output pins, we get gates with many inputs (which are slow).

1.2 Programmable Array Logic

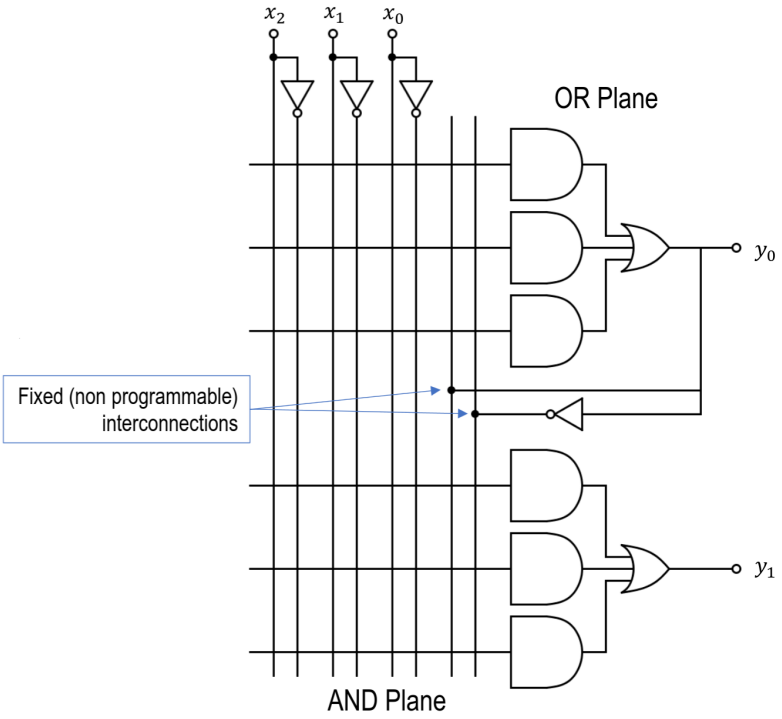
Programmable Array Logic (**PLA**) is yet another approach that aims to increase speed by reducing flexibility.

The idea is to reduce the number of programmable connections, by reducing parasitic resistances and capacitances. This makes devices faster than ROMs and PLAs.

We then settle on *programmable AND, fixed OR*, with the AND plane identical to PLA and a fixed OR plane.

There are limits the number of implicants for each function, regardless of the total number of AND gates in the AND plane, so a restricted set of combinational functions that can be implemented compared to ROM and PLA.

Let's look at the structure:



From the figure we notice:

- The input NOT plane, giving affirmed and complemented inputs;
- Grouped AND gates, with programmable (not shown in figure) interconnects to the inputs;
- Fixed output OR plane connections, *but* with output feedbacks before the AND plane, alongside inputs. This allows to implement functions using multi-level logic.

1.2.1 PAL performance

PAL has a few probelms. First of, fewer functions can be implemented using a PAL with respect to a PLA, and outputs cannot share implicants among functions.

Feedback connections may help, but they use more resources, making for higher costs and delays.

Also, sequential logic cannot be implemented directly: feedbacks can be used in principle to implement sequential logic, but they are too slow to be useful.

1.3 Macrocells and macroblocks

A **macrocell** is a configurable logic circuit made of a PLA or a PAL, with a flip-flop added to each output. Multiplexers are used to select, either via:

- Combinational bypass (direct and inverted);
- Latched output (through the flip-flops, direct and inverted).

Only the selection inputs of the multiplexers can be configured, not the data inputs or outputs, and they are configured using the same technology as the combinational logic.

A **macroblock** is a collection of macrocells, which share all inputs and PAL feedback.